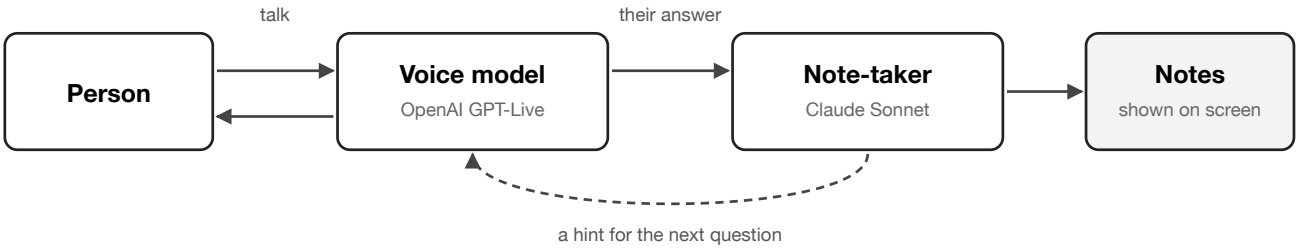
How the Viven interviewer works, and why

Utkarsh Kanwat · November 2026 · viven-interviewer.utkarshkanwat.com

The goal is simple to say: talk with someone for twenty minutes, by voice, and come away with notes good enough for a digital twin to answer the way they would. It should work for anyone, not just people who are good at explaining themselves. Five things shaped the design. Each one started as a mistake.



The voice model does the talking. The note-taker listens, writes the notes, and suggests where to go next.

1 The voice model talks, the note-taker listens

At first the note-taker wrote every question and the voice model read it out. Replies took two to eight seconds, and sometimes the voice model said its own line and then the scripted one, so two questions came out at once. Now the voice model answers on its own, in about a second, because it can hear when someone has finished. The note-taker reads each answer, updates the notes, and sends a quiet hint for the next question. It doesn't always take the hint, and that's fine: a late question is worse than a different one.

2 No behaviour rules in code

I added code to stop repeated questions, cap how long it stays on one topic, and trim long questions. Each rule broke a normal conversation somewhere. It cut questions off mid-sentence, blocked a fair follow-up, and left a topic the person was enjoying. I removed them. The prompt now describes what a good interviewer does, and the model decides. Code only moves data and saves it.

3 Notes in the person's own words

The first version saved labelled facts: a value, a rule, a story. The labels didn't help anyone, and a lot of what got saved was about the call ("didn't like being interrupted") rather than the person. Now it keeps a small folder of short notes, such as work, family and views, written in their words. After each call it rereads everything against the transcript and tidies it up.

4 Ask about what makes them different

In my own test call it asked five questions about how I prioritise my work. Every answer was saved, but anyone would have said the same. A twin learns nothing from that. Now it starts with what the person is known for, since that's what people will ask their twin. Then it gets a rough picture of the rest of their life, and only digs in where an answer is unusual. After each answer it privately asks itself whether the topic is covered or what's still missing, and moves on when it's covered.

5 A screen that doesn't feel like a test

The first screen looked like a debugging tool, and that makes people perform. Now you type your name and start talking. The notes appear next to the conversation as they're written, so you can see what it understood. There is one button to end, and when you come back it picks up where you left off.

How I check changes

I don't trust a prompt change until it's tested. Ten simulated people, each with hidden facts about their life, are interviewed by the real system. Afterwards a separate model answers questions about each person using only the notes, and a model from another company grades the answers. The latest round covered more of each person's life (2.8 areas instead of 2.0) without losing any facts. Most big rewrites I tried made things worse. Small, precise wording changes were what helped.

How it differs from TwinCafe's Train

From using TwinCafe's story interview on my own account, 6 November

TwinCafe has the same goal: teach a twin how someone thinks. I used its Train page on my own account: told a story by voice, then asked the twin outsider questions. It's built around topics and recordings. This one is built around a conversation and notes.

	TwinCafe Train	This interviewer
Where it starts	You pick one of 12 topics generated from LinkedIn, all about your work.	No list. One easy question about what you do these days, then it follows your answers.
Changing topic	Stays on the chosen topic. When I asked to switch: "let's wrap this memory thread quickly".	Goes where you go. Sketches the rest of your life before digging into any part of it.
What gets saved	The raw transcript as one document, with speech-to-text errors and call chatter.	Short notes in your words, in folders (work, family, views), checked against the transcript after the call.
Finding gaps	After the fact: someone asks the twin, you see what it missed, then add material.	During the call: after every answer it asks what's missing and steers the next question there.
Coming back	Each story is a separate recording.	"Call me later" picks up from what you said last and the open gaps.
Personal side	On hobbies and my move the twin said "I haven't shared that". On "everything is urgent, what do you do?" it confidently gave a senior-engineer answer that wasn't mine.	Aims at that layer too, once the work side is in, because that's where a twin otherwise falls back to a generic person.

What I took from them. They're right about whose questions matter: the people who will talk to the twin, and they mostly come for the person's work. My early version asked about morning routines as eagerly as about someone's field. Now it starts with what people would ask the twin about, and asks open questions that invite long answers.

What happens during one call

1. **Start.** Type your name and talk. A link (LinkedIn, a website) is optional; it's read as background and marked "not said by them".
2. **Talk.** The voice model (OpenAI GPT-Live over WebRTC) holds the conversation and replies in about a second.
3. **After each answer**, in the background, the note-taker (Claude Sonnet) reads the answer with the notebook and:
   - judges whether the answer was complete and what a fuller one would add,
   - writes, moves or merges notes,
   - keeps a private list of up to 12 gaps, most valuable first,
   - sends the voice model a silent hint for the next question.
4. **On screen**, the notes appear beside the conversation as they're written.
5. **"Call me later"** ends politely. The next call opens by name, mentions something real from last time, and continues from the gaps.
6. **After the call**, the whole notebook is reread against the transcript to fix and fill notes, and a report lists what was learned, what's missing and what to ask next.

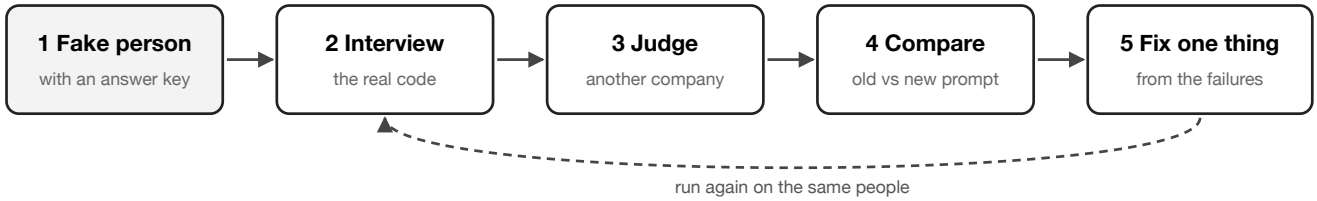
Where it runs

Each person gets their own small server process on Cloudflare (a Durable Object) that handles their calls one at a time and keeps their notes; the list of people is in a Cloudflare database. The same code also runs as a plain Node server with Postgres or files. The notebook fits in the model's context, so there's no search index; past about 100,000 characters the plan is to let the model browse the folders with list, read and search tools.

How the Viven interviewer is tested

The eval harness, eval/ in the repo

I can't know what a real person could have said. So I made people where I do know, and measure how much of it ends up in the notes, whether a twin could answer from them, and what else got worse.



1 A fake person with an answer key

There are 10 made-up people, from Meera, a 58-year-old retired school principal in Pune, to a wary ICU nurse. Each has two things the interviewer never sees:

- **About 10 hidden facts** (104 in all). *Meera: "she judges a school by its toilets, not its results."* Each is tagged by how hard it is to get: shared freely (11), only if asked (37), only if probed (34), only once they trust you (22). Important ones are flagged.
- **5 questions an outsider might ask their twin**, with the right answers. *"How would she handle an angry parent?"*

2 The fake person talks to the real interviewer

A GPT model plays Meera. It's given her facts and personality, and told to give a fact only when the conversation earns it. It talks to the same code the website runs, in text instead of voice.

- Each turn it rates its own annoyance from 1 to 5; at 5 it walks out. After 6 answers it says "call me later" and comes back in a new session, to test the resume.

3 A judge grades it

The judge is from a different company (DeepSeek, falling back to Gemini), so it isn't grading its own kind of writing. It scores:

- **Facts:** for each hidden fact, did Meera say it, and did it reach the notes?
- **Twin test** (the main score): another model gets only the notes and answers the 5 questions; each answer is marked right, partly right, wrong or "don't know".
- **Every interviewer line:** a repeat, two questions in one, praise, pushing after a no?
- **Notes:** is every note backed by something she actually said?

4 Compare two versions

Old and new prompt talk to the same people with the same seed (a number that makes the fake person's random choices repeat), compared person by person.

- Each score comes with a 95% range. If the range crosses zero, the report says "can't tell", not "better".
- Guard scores (walk-outs, annoyance, repeats, made-up answers) must not get worse, even if the main score rises.

5 Read the failures, change one thing, repeat

A viewer shows each conversation beside its grades. I read where it went wrong, make one small prompt edit, and run again. Five people are for finding problems; five are held back to confirm the fix.

Real example: the latest round

The change: move on once a topic has their view and one real case. 5 people × 2 seeds, graded blind.

Score	Before	After
Areas of life covered	2.0	**2.8**
Facts drawn out	0.28	**0.32**
… only once they trust you	0.05	0.09
Re-asked questions (guard)	0.2	**0.5 ✗**

More breadth, but re-asks went up: the "did something else break" check.

Real people, and trusting the judge

- **Real people have no answer key.** The judge writes likely outsider questions, answers them from the notes and checks each against the transcript. Hosted conversations are pulled in every 2 hours.
- Conversations are graded twice to see how often labels flip, and two seeds show how big a change must be to beat luck.
- **Not done yet:** my own hand marks to compare with the judge's, and the run on the five held-back people.